数据结构

树与二叉树

考试大纲

四、树与二叉树

（一）树的基本概念

（二）二叉树

1.二叉树的定义及其主要特征

2.二叉树的顺序存储结构和链式存储结构

3.二叉树的遍历

4.线索二叉树的基本概念和构造

（三）树、森林

1.树的存储结构

2.森林与二叉树的转换

3.树和森林的遍历

（四）树与二叉树的应用

1.哈夫曼（Huffman）树和哈夫曼编码

2.并查集及其应用

3.堆及其应用（去年新增）

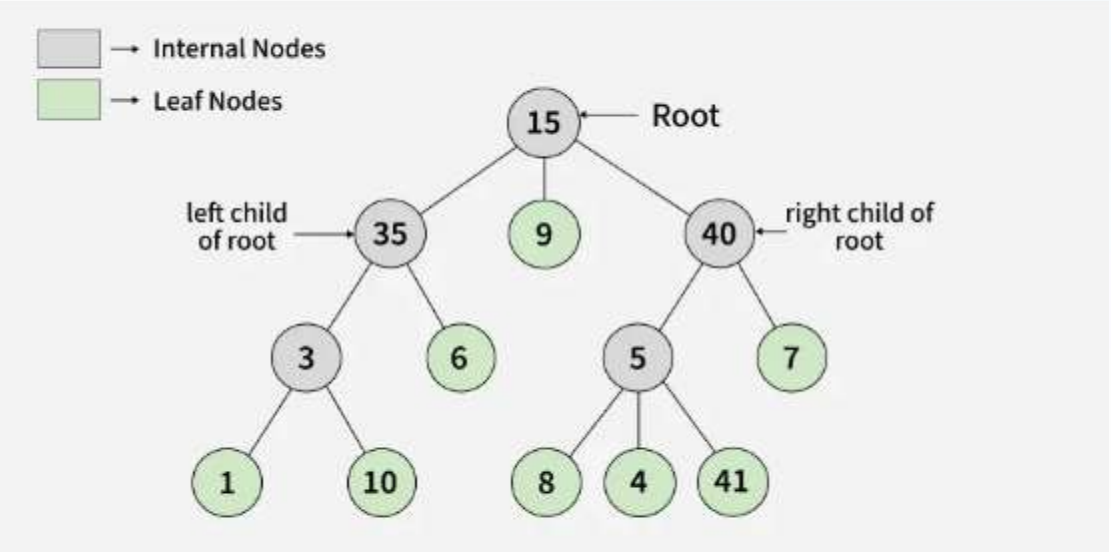
树的递归定义与图解

逻辑特征

树是 n ($n \geq 0$) 个结点的有限集。n=0 为空树。在非空树中：

- 有且仅有一个根结点（Root）。
- 其余结点分为 m 个互不相交的有限集，每一集本身又是一棵树。

> **核心：** 树是一种分层结构，具有“一对多”的关系。



核心术语：度、深度与高度

术语	定义说明
结点的度	该结点拥有的子树个数。
树的度	树中所有结点度数的最大值。
结点的深度	自顶向下计算（根为1）。
结点的高度	自底向上计算。
叶子结点	度为 0 的结点（终端结点）。

树的基本数学性质

公式 1

树中的结点总数等于所有结点的度数之和加 1。

$$n = \sum d_i + 1$$

公式 2

度为 m 的树中第 i 层上至多有 m^{i-1} 个结点 ($i \geq 1$)。

二叉树的定义与特殊状态

关键特征

1. 每个结点至多有两棵子树。
2. 子树有左右之分，次序不能颠倒。

> 注意：即使某结点只有一个子树，也要区分左/右子树。

五种基本形态

- 空二叉树
- 仅有根结点
- 右子树为空
- 左子树为空
- 左右子树均非空

满二叉树与完全二叉树

满二叉树 (Full)

高度为 h 且含有 $2^h - 1$个结点的二叉树。叶子全部在最后一层。

完全二叉树 (Complete)

高度为 h，前 h-1 层为满，第 h 层结点连续集中在左侧。

核心：编号逻辑与存储效率最优。

二叉树性质：$n_0 = n_2 + 1$

核心定理（高频选择题考点）

对任何非空二叉树，若叶子结点数为 n_0，度为 2 的结点数为n_2，则：

$$n_0 = n_2 + 1$$

推导提示：结点总数 $n = n_0 + n_1 + n_2$，且 $n =$ 边数 $+ 1 = (n_1 + 2n_2) + 1$。

完全二叉树的编号性质

若对完全二叉树按层序编号 (1...n)

父结点与孩子	结点 i 的左孩子为 $2i$，右孩子为 2i+1，父结点为 $\lfloor i/2 \rfloor$。
叶子结点范围	若$i > \lfloor n/2 \rfloor$，则 i 为叶子结点。
高度计算	$h = \lfloor \log_2 n \rfloor + 1$

二叉树的顺序存储

适用场景

主要用于**完全二叉树**。对于一般二叉树，会造成严重的存储空间浪费（空位需占位）。

> **优势：** 无需存储指针，逻辑关系隐式包含在数组下标中。

数组下标：A[1...n]

这是**堆 (Heap)** 的物理基础。

二叉树的链式存储：二叉链表

二叉链表结构

每个结点含三个域：[lchild | data | rchild]

> 在 n 个结点的二叉链表中，含有 **n+1** 个空链域。

物理实现更灵活，是大多数树算法的标准形式。

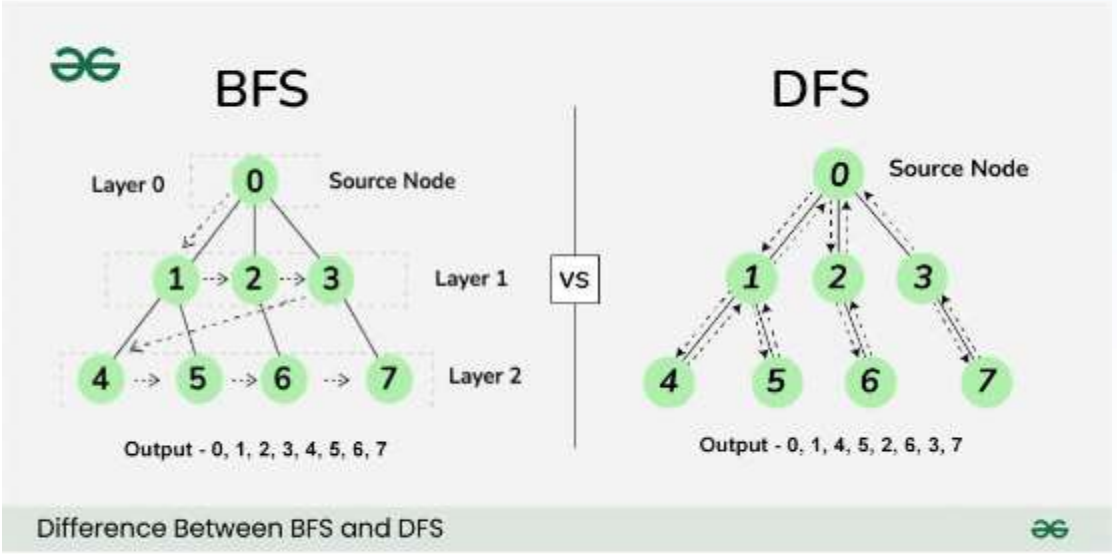
```
typedef struct BiTNode{
    ElemType data;                          //数据域
    struct BiTNode *lchild,*rchild;         //分别指向左右孩子的指针
}BiTNode,*BiTree;
```

二叉树的遍历：先、中、后序

遍历逻辑

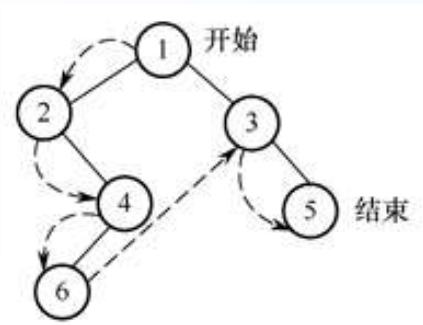
- **先序 (DLR)：** 根 -> 左 -> 右
- **中序 (LDR)：** 左 -> 根 -> 右
- **后序 (LRD)：** 左 -> 右 -> 根

> **重构准则：** 中序 + (先序或后序) 序列可以唯一确定一棵二叉树。

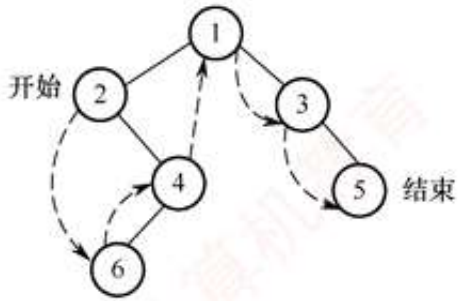


二叉树的遍历：先、中、后序

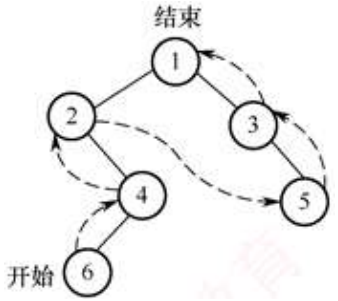
```
void PreOrder(BiTree T){
    if(T!=NULL){
        visit(T);                    //访问根结点
        PreOrder(T->lchild);         //递归遍历左子树
        PreOrder(T->rchild);         //递归遍历右子树
    }
}
```




```
void InOrder(BiTree T){
    if(T!=NULL){
        InOrder(T->lchild);          //递归遍历左子树
        visit(T);                    //访问根结点
        InOrder(T->rchild);          //递归遍历右子树
    }
}
```




```
void PostOrder(BiTree T){
    if(T!=NULL){
        PostOrder(T->lchild);        //递归遍历左子树
        PostOrder(T->rchild);        //递归遍历右子树
        visit(T);                    //访问根结点
    }
}
```



层序遍历与队列实现

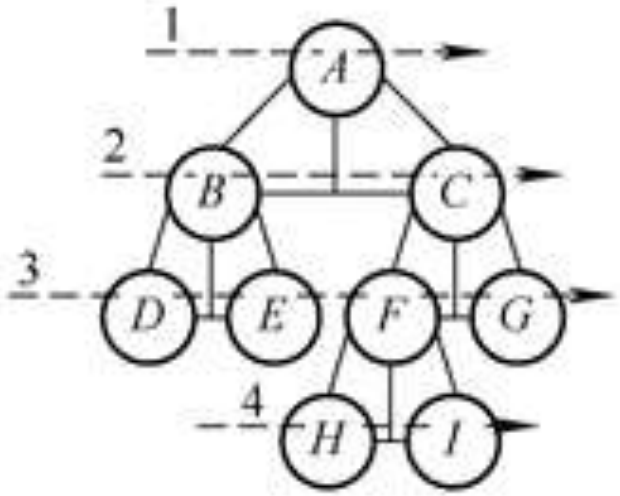
算法步骤

1. 初始化一个队列，根结点入队。
2. 若队不空，出队一个结点访问。
3. 若该结点有左孩子，左孩子入队。
4. 若该结点有右孩子，右孩子入队。
5. 重复 2-4 直到队空。

层序遍历是解决**二叉树宽度**、**完全性判定**问题的关键。

层序遍历与队列实现

```
void LevelOrder(BiTree T){
    InitQueue(Q);                          //初始化辅助队列
    BiTree p;
    EnQueue(Q,T);                          //将根结点入队
    while(!IsEmpty(Q)){                    //队列不空则循环
        DeQueue(Q, p);                     //队头结点出队
        visit(p);                          //访问出队结点
        if(p->lchild!=NULL)
            EnQueue(Q,p->lchild);          //若左孩子不空，则左孩子入队
        if(p->rchild!=NULL)
            EnQueue(Q,p->rchild);          //若右孩子不空，则右孩子入队
    }
}
```



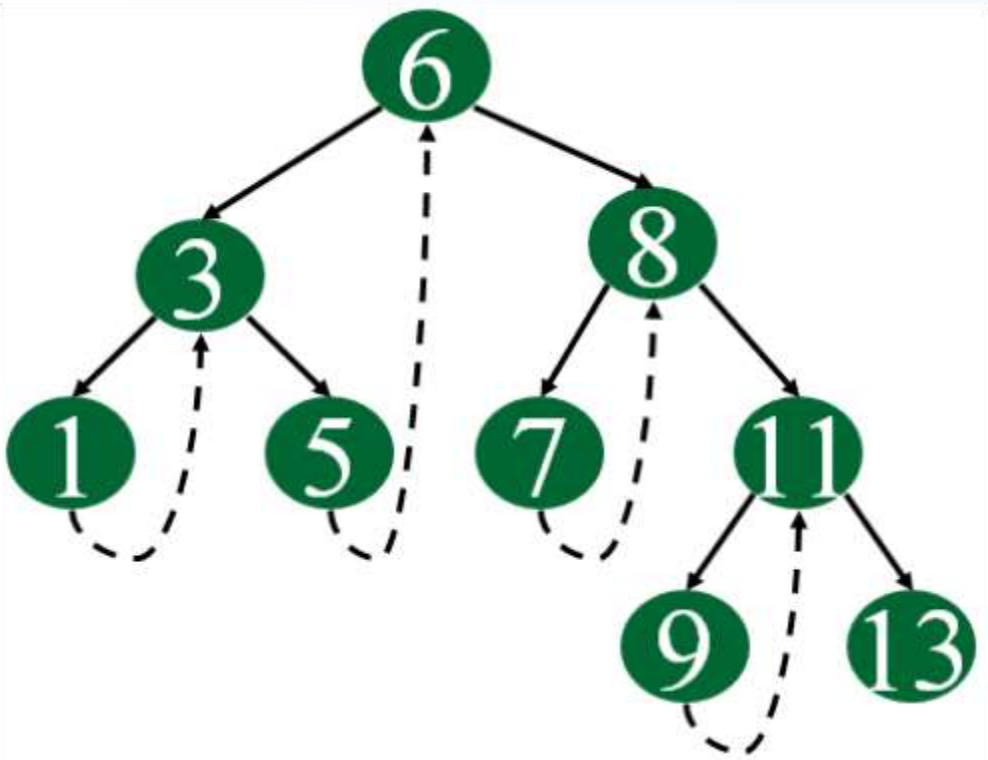
线索二叉树的基本概念

引入动机

利用 n+1 个空链域存放结点的**前驱**和**后继**信息，加速查找。

- ltag=0: 指向左孩子
- ltag=1: 指向前驱

```
typedef struct ThreadNode{
    ElemType data;                          //数据元素
    struct ThreadNode *lchild,*rchild;      //左右孩子指针
    int ltag,rtag;                          //左右线索标志
}ThreadNode,*ThreadTree;
```



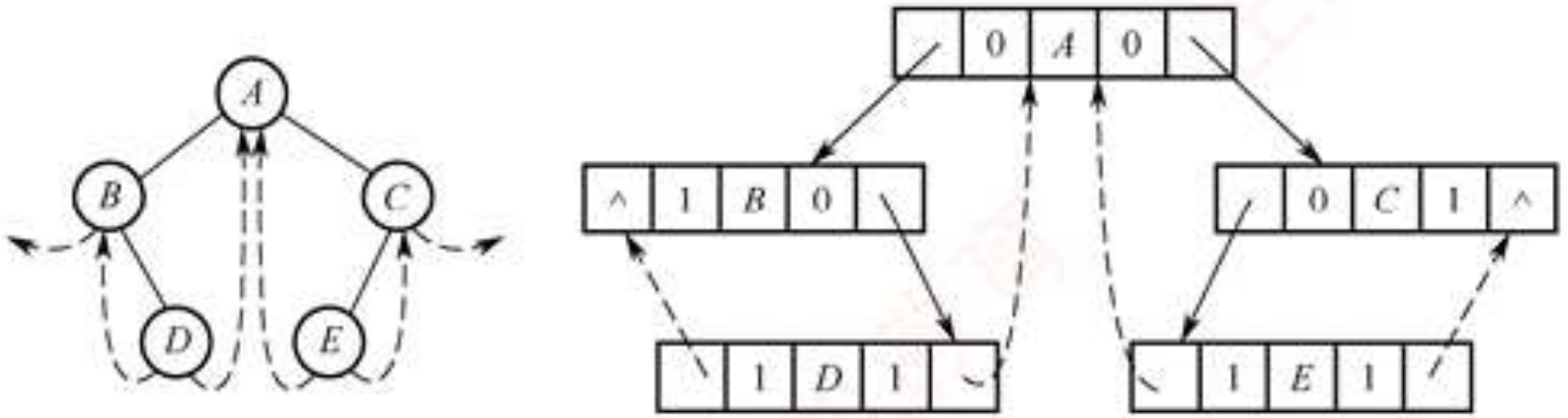
中序线索二叉树的查找

找后继

若 rtag=1，后继为 rchild。若 rtag=0，后继为右子树中最左侧结点。

找前驱

若 ltag=1，前驱为 lchild。若 ltag=0，前驱为左子树中最右侧结点。



中序线索二叉树的实现

通过中序遍历对二叉树线索化的递归算法如下：

```
void InThread(ThreadTree &p,ThreadTree &pre){
    if(p!=NULL){
    InThread(p->lchild,pre);        //递归，线索化左子树
    if(p->lchild==NULL){            //当前结点的左子树为空
        p->lchild=pre;              //建立当前结点的前驱线索
        p->ltag=1;
    }
    if(pre!=NULL&&pre->rchild==NULL){//前驱结点非空且其右子树为空
        pre->rchild=p;              //建立前驱结点的后继线索
        pre->rtag=1;
    }
    pre=p;                          //标记当前结点成为刚刚访问过的结点
    InThread(p->rchild,pre);        //递归，线索化右子树
  }
}
```

建立中序线索二叉树的主过程如下：

```
void CreateInThread(ThreadTree T){
    ThreadTree pre=NULL;
    if(T!=NULL){                    //非空二叉树，线索化
        InThread(T,pre);            //线索化二叉树
        pre->rchild=NULL;           //处理遍历的最后一个结点
        pre->rtag=1;
    }
}
```

树的存储：孩子兄弟表示法

> **左孩子右兄弟：** 每个结点含两个指针，左指向第一个孩子，右指向下一个兄弟。

物理形态

[firstchild | data | nextsibling]

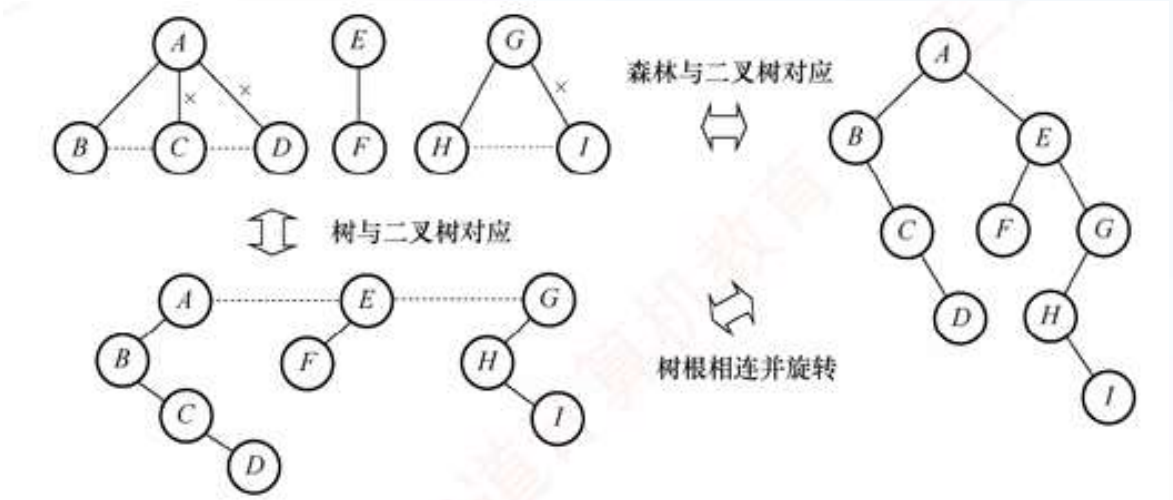
这种存储方式在物理结构上与**二叉链表**完全一致，是树与二叉树转换的基础。

树、森林与二叉树的转换

转换核心

1. 相邻兄弟连线。
2. 对每个结点，保留与首个孩子的连线，抹除与其他孩子的连线。

> 森林转换：各树根结点视为兄弟。



遍历方式的对应关系

树/森林遍历	对应二叉树遍历
树的先根遍历	先序遍历
树的后根遍历	**中序遍历**
森林的先序遍历	先序遍历
森林的中序遍历	**中序遍历**

哈夫曼树 (最优二叉树)

核心指标：WPL

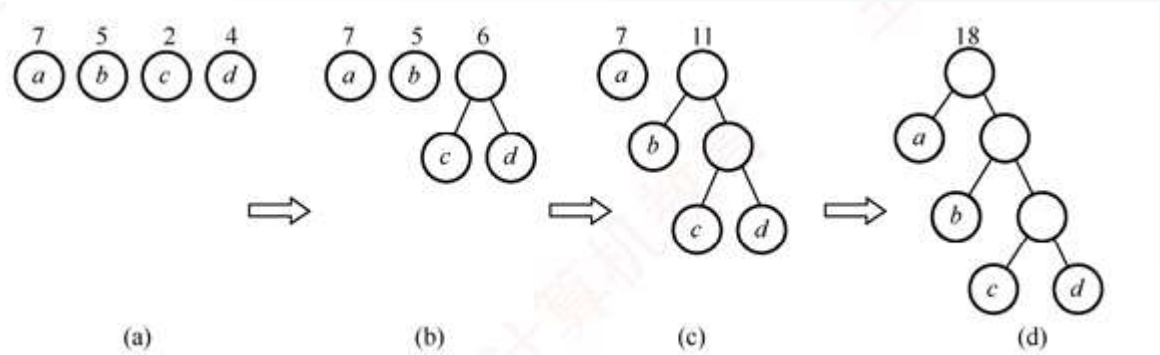
树中所有叶子结点的**带权路径长度**之和。

$$WPL = \sum_{i=1}^{n} w_i l_i$$

WPL 最小的二叉树即为哈夫曼树。

哈夫曼树构造算法步骤

1. 将 n 个结点看作 n 棵独立的树。
2. 选取两棵**权值最小**的树作为子树，构造新根，新权值为两者之和。
3. 重复直到只剩一棵树。



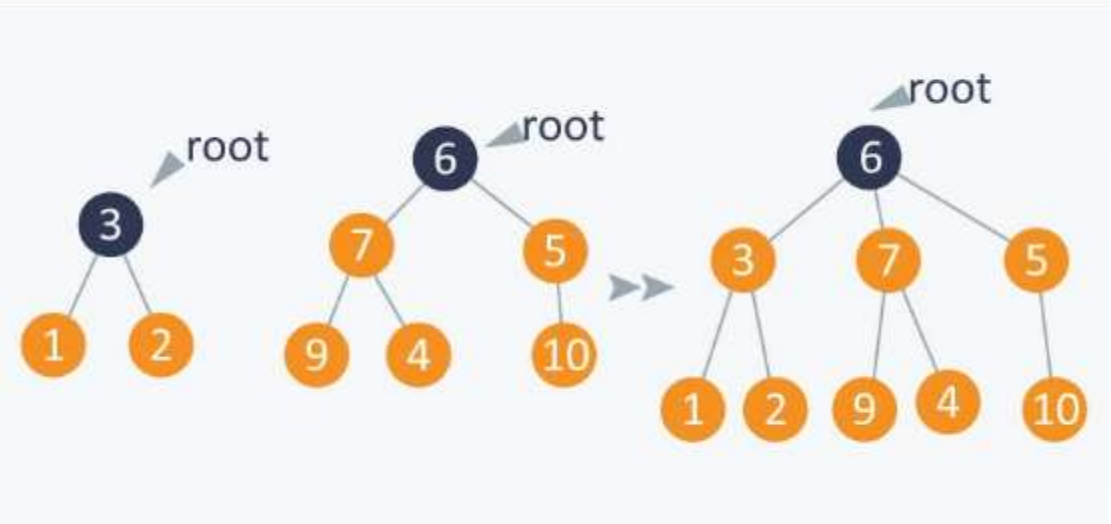
哈夫曼树的三个重要性质

- 哈夫曼树中不存在度为 1 的结点。
- 若有 n 个叶子结点，总结点数为 2n-1。
- 生成的编码是**前缀编码**，解码无二义性。

并查集 (Union-Find) 逻辑

用于处理不相交集合的合并与查询问题。

- Find: 查找所属集合（根结点）。
- Union: 将两个集合合并。



并查集的数组存储实现

利用**双亲表示法**，数组 S[i] 存放父结点下标。

- 若 S[i] < 0，则 i 是根。
- 其绝对值常表示该集合中结点的总数。

路径压缩： 在 Find 过程中直接将路径上结点挂到根下，提升效率至近乎 O(1)。

并查集的优化

用数组、双亲表示法来描述元素之间的集合关系。根节点为 -1，非根节点指向父节点下标。

Find (int S[], x) ——找到x所属集合
Union(int S[], int x, int y) —— 将x、y所属集合合并

最坏时间复杂度：

Find 操作 = 最坏树高 = O(n)

将n个独立元素通过多次Union合并为一个集合——$O(n^2)$

Union 优化

用根节点的负值表示一棵树的结点总数

每次 Union 操作让小树合并到大树根节点下面

最坏时间复杂度：

Find 操作=最坏树高= $O(\log_2 n)$

将n个独立元素通过多次Union合并为一个集合——$O(n\log_2 n)$

Find 优化

“压缩路径”的策略，每次 Find 操作先找到 x 所属根节点，再将查找路径上的所有结点都直接挂在根节点下面

最坏时间复杂度：

Find 操作=最坏树高= $O(\alpha(n))$

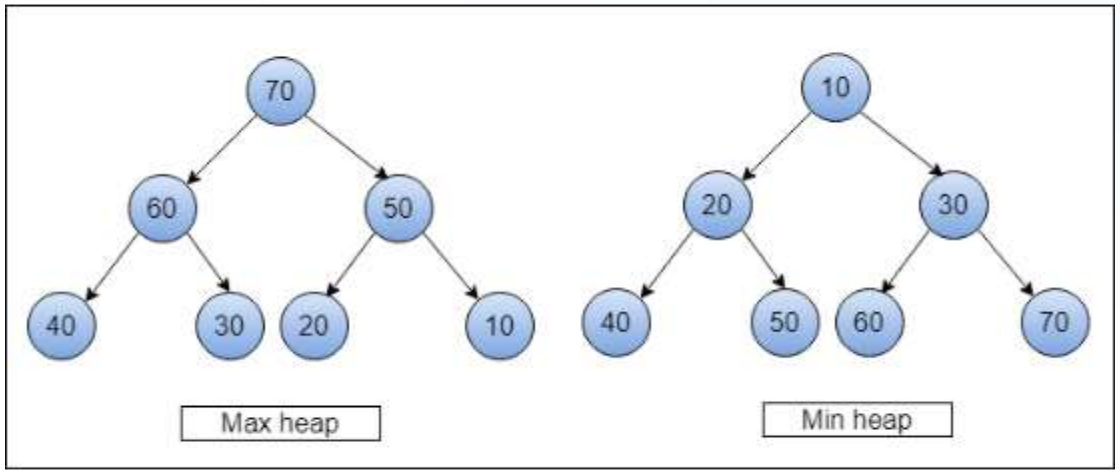
将n个独立元素通过多次Union合并为一个集合——$O(n\,\alpha(n))$

堆 (Heap) 的定义与类型

逻辑结构

堆是一棵**完全二叉树**，且满足：

- **大根堆：** 根 ≥ 左右孩子。
- **小根堆：** 根 ≤ 左右孩子。



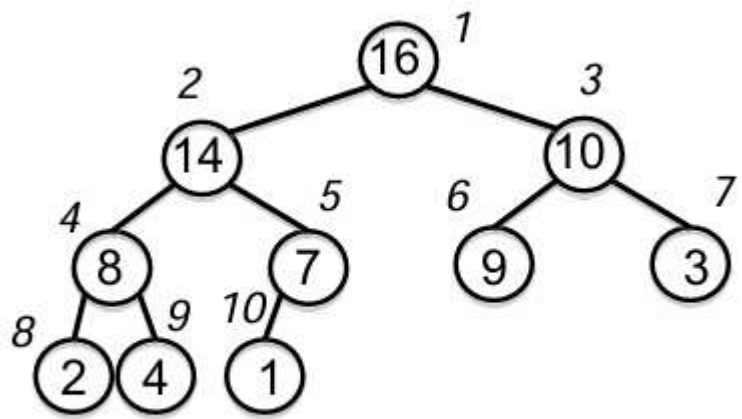
Heap as a Tree

root of tree: first element in the array, corresponding to i = 1

parent(i) =i/2: returns index of node's parent

left(i)=2i: returns index of node's left child

right(i)=2i+1: returns index of node's right child



1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

No pointers required! Height of a binary heap is O(lg n)

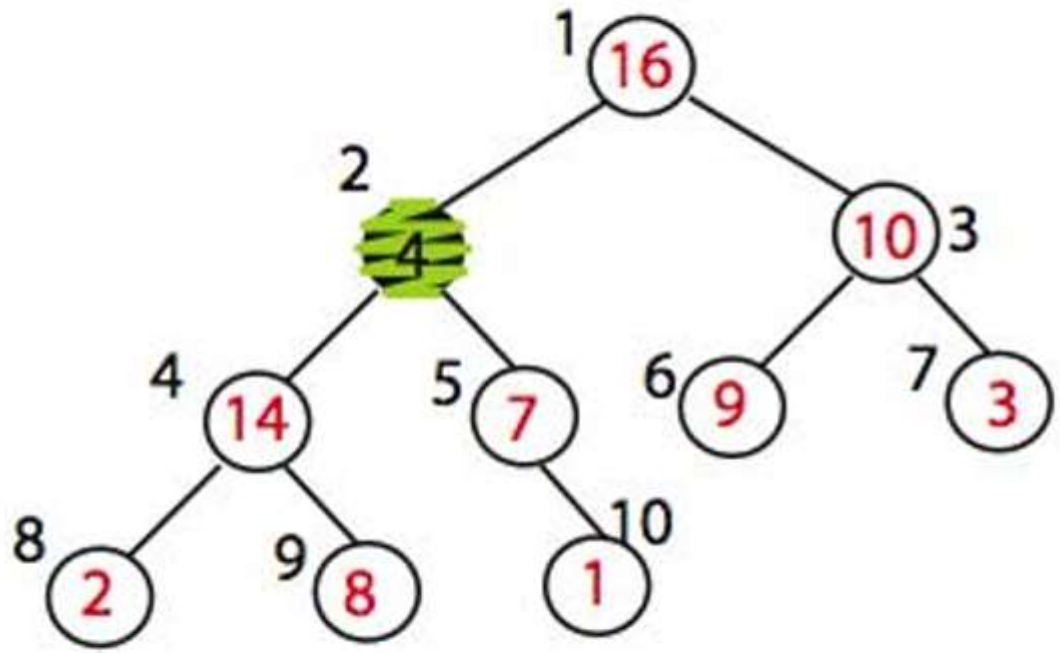
Heap Operations

build_max_heap : produce a max-heap from an unordered array

max_heapify : correct a single violation of the heap property in a subtree at its root

insert, extract_max, heapsort

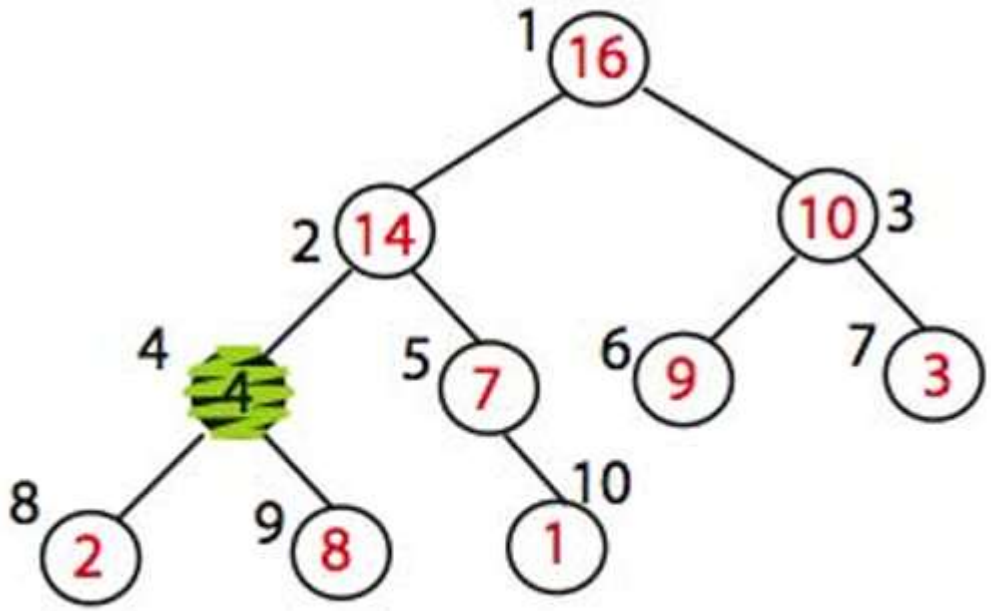
Max_heapify (Example)



MAX_HEAPIFY (A,2)
heap_size[A] = 10

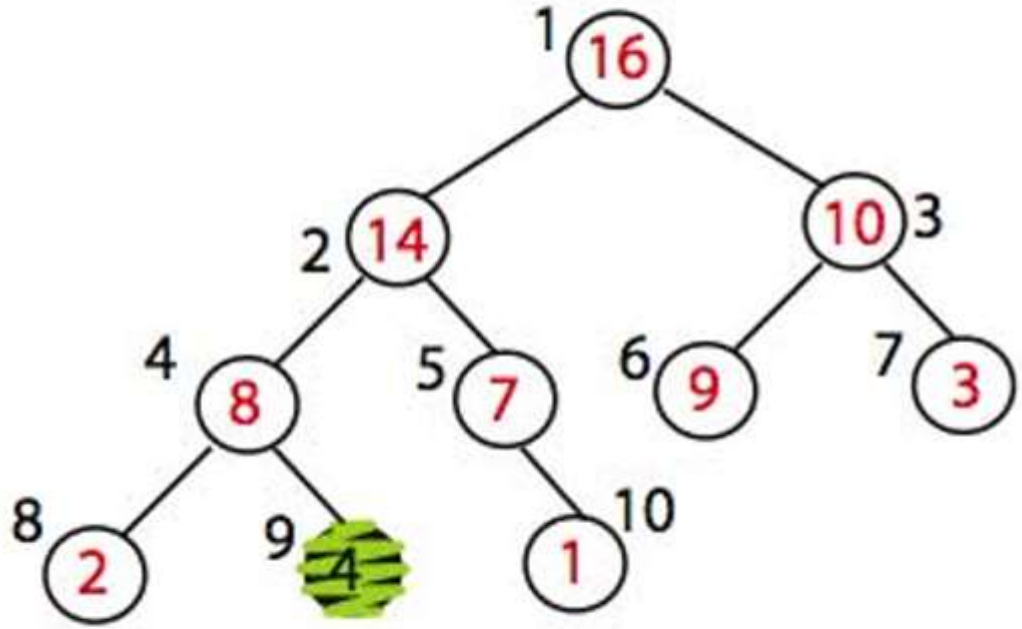
Node 10 is the left child of node 5 but is drawn to the right for convenience

Max_heapify (Example)



Exchange A[2] with A[4]
Call MAX_HEAPIFY(A,4)
because max_heap property
is violated

Max_heapify (Example)



Exchange A[4] with A[9]
No more calls

Time=? O(log n)

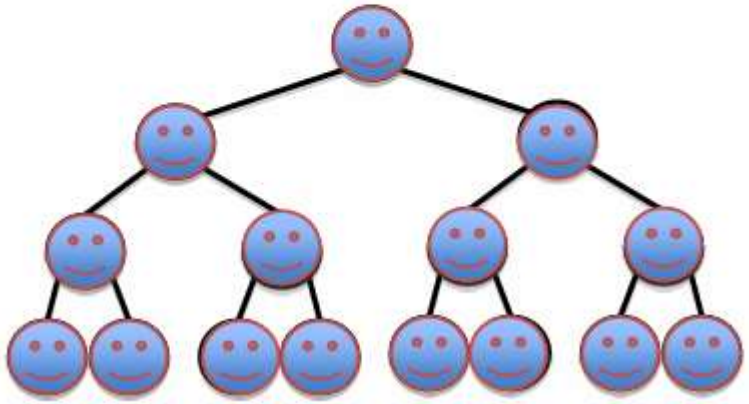
Max_Heapify Pseudocode

```
l = left(i)
r = right(i)
if (l <= heap-size(A) and A[l] > A[i])
    then largest = l     else largest = i
if (r <= heap-size(A) and A[r] > A[largest])
    then largest = r
if largest ≠ i
    then exchange A[i] and A[largest]
            Max_Heapify(A, largest)
```

Build_Max_Heap(A) Analysis

Converts A[1…*n*] to a max heap

```
Build_Max_Heap(A):
    for i=n/2 downto 1
        do Max_Heapify(A, i)
```



Total amount of work in the for loop can be summed as:

$n/4\ (1\ c) + n/8\ (2\ c) + n/16\ (3\ c) + \ldots + 1\ (\lg n\ c)$

Setting $n/4 = 2^k$ and simplifying we get:

$c\ 2^k(1/2^0 + 2/2^1 + 3/2^2 + \ldots (k+1)/2^k)$

The term is brackets is bounded by a constant!

This means that Build_Max_Heap is $O(n)$

建堆：筛选法 (Down)

从最后一个非叶子结点 $\lfloor n/2 \rfloor$开始，自底向上进行筛选：

> 将当前结点与其较大的子结点交换，若交换后导致子树失堆，继续向下调整。

考点： 给定一串数，模拟构建大/小根堆的过程。

堆的插入与删除操作

插入（向上调整）

将新元素放到表尾，依次与其父结点比较并交换。

删除堆顶（向下调整）

将堆顶与表尾元素交换，删除原堆顶，再对新堆顶向下筛选。

10. 【2016 统考真题】若森林 F 有 15 条边、25 个结点，则 F 包含树的个数是（　）。

A. 8　　B. 9　　C. 10　　D. 11

10. C

解法 1：树有一个重要性质，即在 n 个结点的树中有 $n-1$ 条边，“那么对于每棵树，其结点数比边数多 1”。本题森林中的结点数比边数多 10（即 $25-15=10$），显然共有 10 棵树。

解法 2：仔细分析后发现此题也是考查图的性质：生成树和生成森林。对于图的生成树有一个重要的性质，即图中顶点数若为 n，则其生成树含有 $n-1$ 条边。对比解法 1 中树的性质，不难发现两种解法都用到了性质“树中结点数比边数多 1”，后面的分析如解法 1。

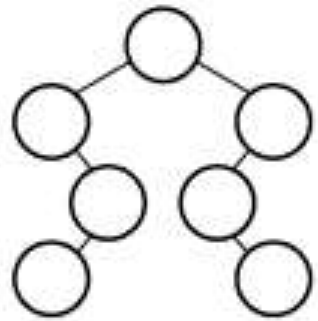
28.【2022 统考真题】若三叉树 T 中有 244 个结点（叶结点的高度为 1），则 T 的高度至少是（　）。

A. 8　　B. 7　　C. 6　　D. 5

28. C

高度一定的三叉树中结点数最多的情况是满三叉树。高度为 5 的满三叉树的结点数 $=3^0+3^1+3^2+3^3+3^4=121$，高度为 6 的满三叉树的结点数 $=3^0+3^1+3^2+3^3+3^4+3^5=364$。由于三叉树 T 的结点数为 244，$121<244<364$，因此 T 的高度至少为 6。

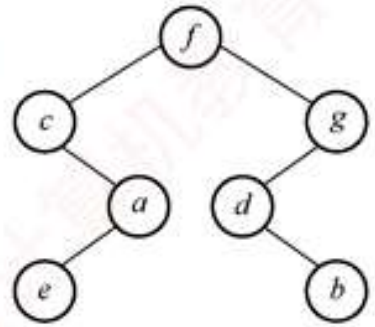
39. 【2017 统考真题】某二叉树的树形如右图所示，其后序序列为 e, a, c, b, d, g, f，树中与结点 a 同层的结点是（　）。



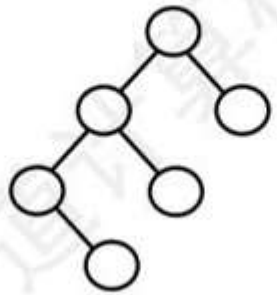
A. c　　B. d　　C. f　　D. g

39. B

后序序列先访问左子树，接着访问右子树，最后访问父结点，递归进行。根结点左子树的叶结点首先被访问，它是 e。接下来是它的父结点 a，然后是 a 的父结点 c。接着访问根结点的右子树。它的叶结点 b 首先被访问，然后是 b 的父结点 d，再后是 d 的父结点 g，最后是根结点 f，如右图所示。因此 d 与 a 同层，选项 B 正确。



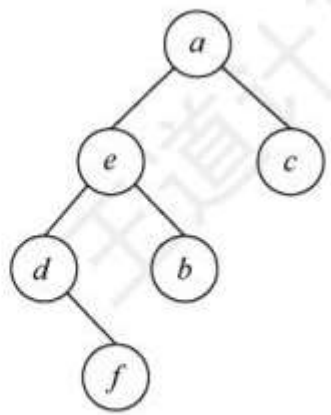
43. 【2023 统考真题】已知一棵二叉树的树形如下图所示，若其后序遍历序列为 *fdbeca*，则其先（前）序遍历序列是（ ）。



A. *aedfbc*　　B. *acebdf*　　C. *cabefd*　　D. *dfebac*

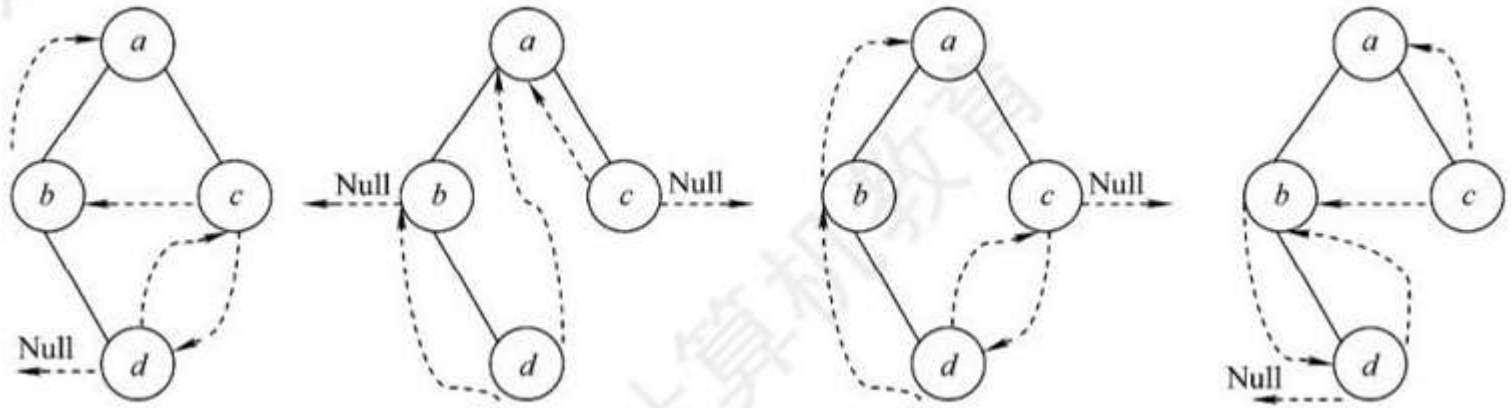
43. A

根据二叉树的树形和后序遍历序列，可以轻松地将各字母填入结点中，如下图所示。



然后对该二叉树进行先序遍历，得到序列 *aedfbc*。

34.【2010 统考真题】下列线索二叉树中（用虚线表示线索），符合后序线索树定义的是（ ）。



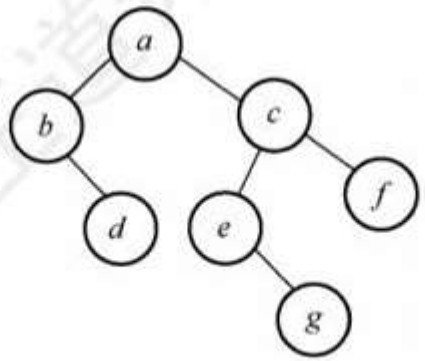
题中所给二叉树的后序序列为 *dbca*。结点 *d* 无前驱和左子树，左链域空，无右子树，右链域指向其后继结点 *b*；结点 *b* 无左子树，左链域指向其前驱结点 *d*；结点 c 无左子树，左链域指向其前驱结点 *b*，无右子树，右链域指向其后继结点 *a*。

19. 【2021 统考真题】某森林 F 对应的二叉树为 T，若 T 的先序遍历序列是 a, b, d, c, e, g, f，中序遍历序列是 b, d, a, e, g, c, f，则 F 中树的棵数是（　）。

A. 1　　B. 2　　C. 3　　D. 4

19. C

由二叉树 T 的先序序列和中序序列可以构造出 T，如下图所示。由森林转化成二叉树的规则可知，森林中每棵树的根结点以右子树的方式相连，所以 T 中的结点 a、c、f 为 F 中树的根结点，森林 F 中有 3 棵树。

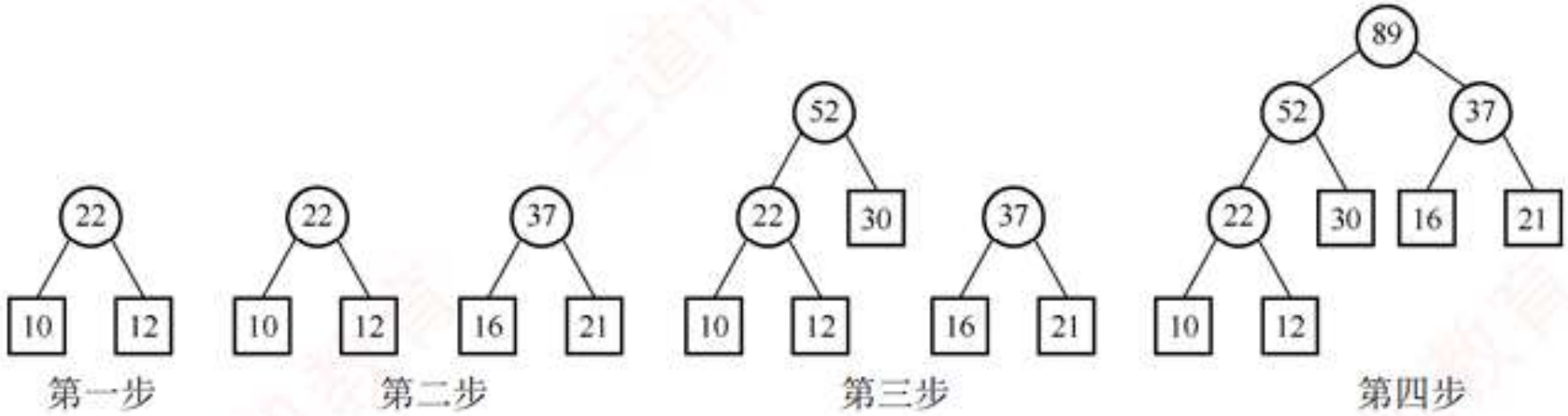


20. 【2021 统考真题】若某二叉树有 5 个叶结点，其权值分别为 10, 12, 16, 21, 30，则其最小的带权路径长度（WPL）是（ ）。

A. 89　　B. 200　　C. 208　　D. 289

20. B

对于带权值的结点，构造出哈夫曼树的带权路径长度（WPL）最小，哈夫曼树的构造过程如下图所示。求得其 WPL = (10 + 12)×3 + (30 + 16 + 21)×2 = 200。

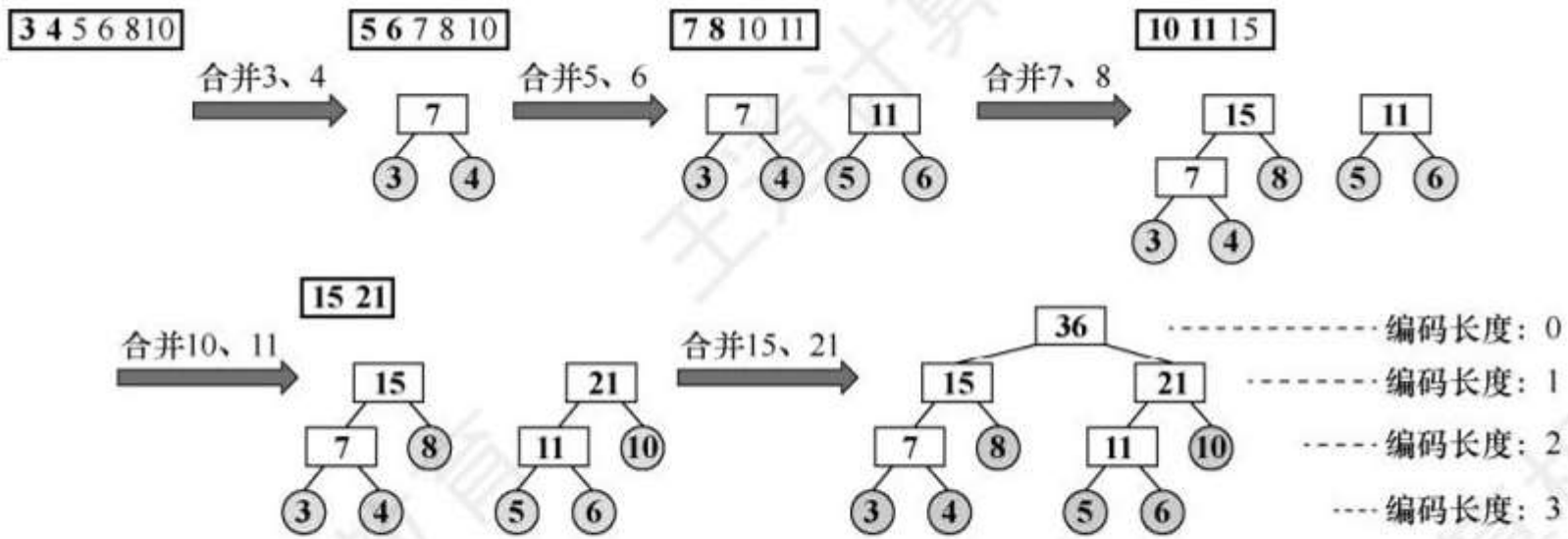


22.【2023 统考真题】在由 6 个字符组成的字符集 S 中，各字符出现的频次分别为 3, 4, 5, 6, 8, 10，为 S 构造的哈夫曼编码的加权平均长度为（　）。

A. 2.4　　B. 2.5　　C. 2.67　　D. 2.75

22. B

构建哈夫曼树的过程如下图所示。



对叶结点的哈夫曼编码，共有 4 个长度为 3 的叶结点、2 个长度为 2 的叶结点，编码的加权平均长度为$[(3+4+5+6)\times3+(8+10)\times2]/(3+4+5+6+8+10)=2.5$。

11. 并查集中最核心的两个操作是：① 查找，查找两个元素是否属于同一个集合；② 合并，若两个元素不属于同一个集合，且所在的两个集合互不相交，则合并这两个集合。假设初始长度为 10（0～9）的并查集，按 1-2、3-4、5-6、7-8、8-9、1-8、0-5、1-9 的顺序进行查找和合并操作，最终并查集共有（ ）个集合。

A. 1　　B. 2　　C. 3　　D. 4

11. C

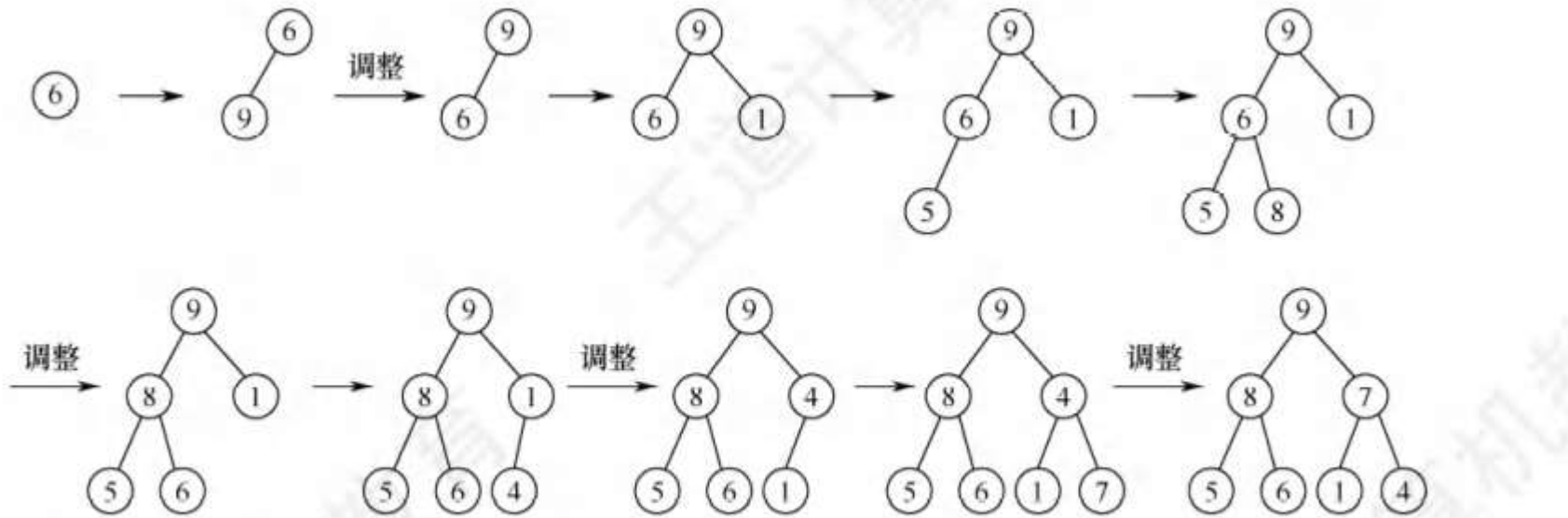
初始时，0～9 各自成一个集合。查找 1-2 时，合并{1}和{2}；查找 3-4 时，合并{3}和{4}；查找 5-6 时，合并{5}和{6}；查找 7-8 时，合并{7}和{8}；查找 8-9 时，合并{7, 8}和{9}；查找 1-8 时，合并{1, 2}和{7, 8, 9}；查找 0-5 时，合并{0}和{5, 6}；查找 1-9 时，它们属于同一个集合。最终的集合为{0, 5, 6}、{1, 2, 7, 8, 9}和{3, 4}，因此答案选择选项 C。

20.【2021 统考真题】将关键字 6,9,1,5,8,4,7 依次插入初始为空的大根堆 H，得到的 H 是(　)。

A. 9,8,7,6,5,4,1　　B. 9,8,7,5,6,1,4　　C. 9,8,7,5,6,4,1　　D. 9,6,7,5,8,4,1

20. B

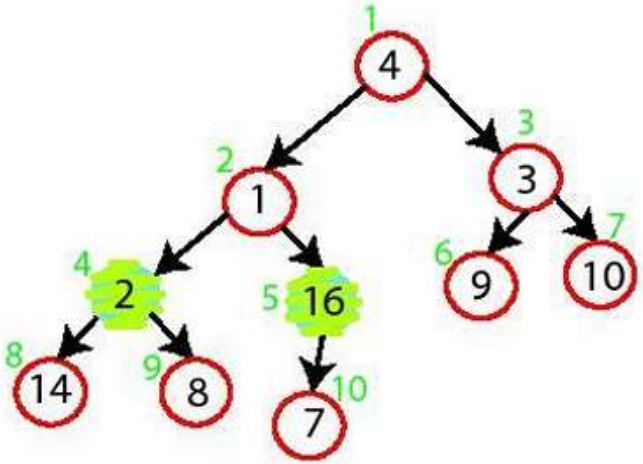
要熟练掌握调整堆的方法，建堆的过程如下图所示，所以答案选择选项 B。



注意，给定序列依次插入空堆的结果与给定序列直接调整成堆的结果是不同的，最终得到的堆的形式也不同。若对序列 6, 9, 1, 5, 8, 4, 7 直接调整成堆，则会误选 C 项。

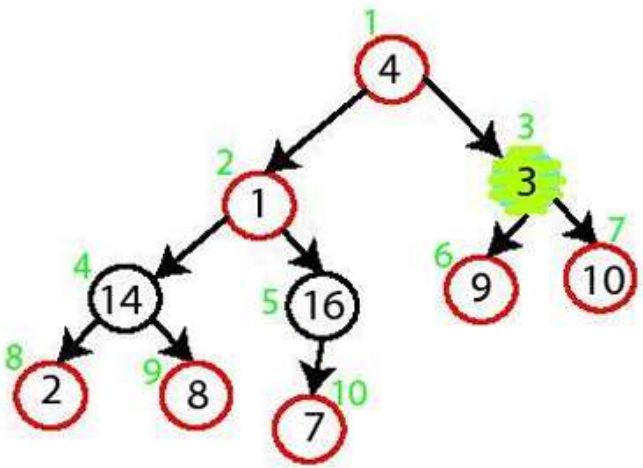
Build-Max-Heap

A	4	1	3	2	16	9	10	14	8	7

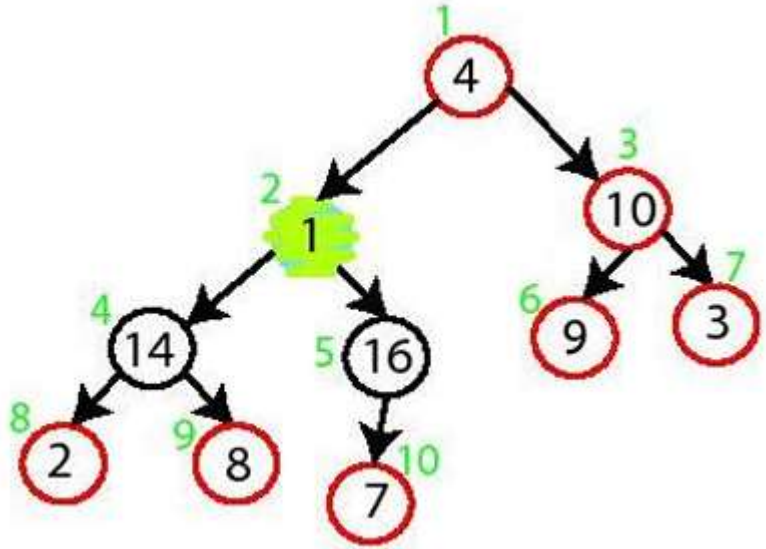


A	4	1	3	2	16	9	10	14	8	7

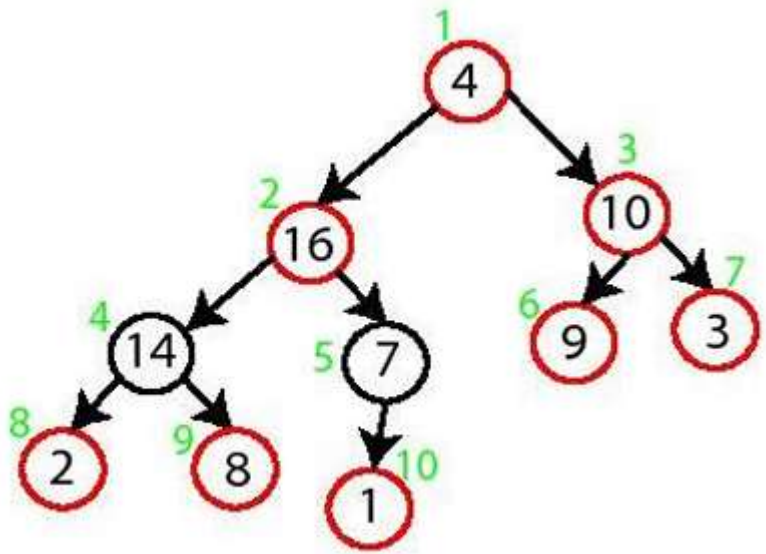
MAX-HEAPIFY (A,5)
no change
MAX-HEAPIFY (A,4)
Swap A[4] and A[8]



MAX-HEAPIFY (A,3)
Swap A[3] and A[7]



MAX-HEAPIFY (A,2)
Swap A[2] and A[5]
Swap A[5] and A[10]



MAX-HEAPIFY (A,1)
Swap A[1] with A[2]
Swap A[2] with A[4]
Swap A[4] with A[9]

A

4	1	3	2	16	9	10	14	8	7

